



RAPPORT DE STAGE

Marissa Lamri

Sommaire

1. Présentation de l'entreprise.....	4
1.1 Histoire et positionnement de Visual Vibes.....	4
1.2 Une approche technique au cœur du projet.....	5
1.3 De la page blanche à l'appli qui tourne.....	5
2. Projet principal : Application web de gestion de recettes.....	7
2.1 Contexte et objectifs du projet.....	7
2.4 Conception technique : schéma de base de données.....	10
3. Mise en place du projet et développement technique.....	11
3.1 Structuration du projet Django.....	11
3.2 Développement des fonctionnalités principales.....	12
3.3 Sécurité et intégrité des données.....	14
3.4 Frontend et expérience utilisateur.....	14
3.5 Gestion du code avec Git et Github.....	15
3.6 Problèmes rencontrés et solutions.....	16
4. Relations hiérarchiques et sociales.....	17
5. Conclusion et perspectives.....	18
5.1 Prolongement du stage.....	19
5.2 Perspectives d'avenir.....	19
Annexes :.....	22

1. Présentation de l'entreprise

1.1 Histoire et positionnement de Visual Vibes

Visual Vibes, c'est une boîte créée par **Charlotte Toussaint** et **Farouk Lamri**, un couple qui baigne dans l'IT et le design. Chacun a sa spécialité, mais ensemble, ils forment un duo ultra complémentaire : l'une manie le branding et l'expérience utilisateur, l'autre gère la technique, le back, le dev... Résultat ? Ils proposent un service 360°, du logo jusqu'au site web qui tourne tout seul.

Quand on pense à une entreprise de design, on imagine souvent des moodboards, des palettes de couleurs, et des typos bien choisies. Mais derrière l'univers visuel de Visual Vibes, il y a aussi du code, des fonctionnalités, de la structure, et un véritable travail d'architecte du web.

Visual Vibes, c'est une petite boîte (mais avec de grandes idées) qui accompagne les créateurs et les indépendants dans tout ce qui touche à leur image professionnelle. Mais ce que je ne savais pas, c'est à quel point la partie développement était importante pour que tout ça fonctionne : site web, tunnel d'achat, espace client, automatisations, sécurité, performances... bref, tout ce qu'on ne voit pas, mais sans quoi rien ne marche. Mon stage s'est déroulé en coulisses, là où se construit ce que l'utilisateur final ne voit pas toujours : interfaces interactives, logique backend, base de données, sécurité... J'ai découvert comment donner vie à un design statique, comment transformer une maquette Figma en interface dynamique, et surtout comment chaque ligne de code peut contribuer à l'expérience utilisateur.

1.2 Une approche technique au cœur du projet

Le terme "service technique" évoque souvent un environnement très formel, structuré autour du code et des infrastructures. Chez Visual Vibes, il prend une forme différente : plus souple, plus humaine, et surtout polyvalente. Chaque tâche, chaque mission est pensée pour s'adapter aux besoins réels des clients, avec une grande autonomie dans les méthodes comme dans les outils utilisés.

Ce pôle, c'est avant tout Farouk, associé de Visual Vibes. Là où Charlotte imagine les interfaces, les expériences et les univers graphiques, lui construit l'infrastructure, la logique et les fonctionnalités. Son rôle va bien au-delà du simple développement : il conçoit des architectures web, sécurise les données, automatise les déploiements, optimise les performances et s'assure que chaque pixel du design s'appuie sur des fondations solides.

1.3 De la page blanche à l'appli qui tourne

Le service technique de Visual Vibes, principalement incarné par Farouk, couvre un spectre large de compétences techniques avancées, bien au-delà du simple développement d'applications. Il intervient à tous les niveaux de la chaîne technique, depuis la conception jusqu'au déploiement sécurisé, avec une forte composante d'architecture cloud, d'automatisation, et de recherche technique continue.

Une de ses activités principales consiste à gérer l'automatisation des processus de déploiement pour les projets clients. Il met en place des pipelines CI/CD, configure des environnements, et supervise la gestion de clusters Kubernetes, que ce soit pour ses propres projets ou ceux déployés chez les clients. Il veille à ce que chaque application soit correctement isolée, scalable, et maintenable dans le temps.

Il prend également en charge toute la sécurité des environnements : contrôle d'accès aux ressources via RBAC, audit des images Docker, mise en place de politiques pour éviter les failles dans les applications déployées, vérification des permissions et surveillance des configurations sensibles.

Une autre partie essentielle de son activité repose sur la gestion des bases de données (modélisation, optimisation, sauvegarde, monitoring) et l'accompagnement des clients dans leurs choix techniques. Farouk intervient régulièrement en amont d'un projet, pour définir l'architecture technique nécessaire, réaliser des POC (Proof of Concept), tester de nouvelles technologies ou proposer des alternatives plus adaptées.

Ce travail d'expérimentation et de conseil permet à Visual Vibes d'aller au-delà du simple livrable technique : il s'agit d'un véritable partenariat avec le client final, dans lequel le service technique joue un rôle de guide, de garant de la fiabilité, mais aussi de force de proposition.

Enfin, Farouk assure également la gestion de projets techniques en interne : il structure les tâches, coordonne les priorités, choisit les outils, et garantit que chaque livrable respecte à la fois les standards de qualité et les contraintes de sécurité ou de performance attendues.

2. Projet principal : Application web de gestion de recettes

2.1 Contexte et objectifs du projet

Le projet principal de mon stage consistait à développer une application web de gestion de recettes, depuis la première idée jusqu'à une version fonctionnelle. L'objectif n'était pas seulement de coder, mais de mener un projet complet, avec toutes ses étapes de conception, de structuration et de réalisation.

Dès le départ, je me suis posé les questions essentielles : que doit contenir une recette ? Comment l'utilisateur doit-il interagir avec l'application ? Cette démarche m'a poussée à réfléchir en amont avant de passer au code, ce qui m'a permis de mieux anticiper les besoins et les contraintes.

Ce projet m'a offert une immersion réelle dans la vie d'un développeur : organisation du code, gestion des erreurs, sécurité des accès, interface responsive... Il m'a permis de progresser autant en technique qu'en autonomie et en méthodologie.

2.2 Choix techniques et outils utilisés

Pour ce projet, le langage imposé était Python, largement reconnu pour sa simplicité et sa polyvalence. Sa syntaxe claire m'a permis de travailler efficacement, tandis que son adoption massive dans le monde professionnel en fait une compétence recherchée.

Ce choix était d'autant plus logique qu'en juin 2025, Python figure une nouvelle fois en tête du classement TIOBE des langages les plus utilisés au monde, avec 25,87 % de part d'usage, soit une croissance de +10,48 % par rapport à l'année précédente. [Voir annexe 1.](#)

Python est utilisé dans le développement web, la data science, le machine learning, l'automatisation, la cybersécurité, etc. Il est également hautement demandé sur le marché du travail, avec des offres dans des domaines aussi variés que la tech, la finance, la santé ou l'éducation.

Il permet aussi de progresser rapidement grâce à sa syntaxe claire, et de rejoindre un écosystème riche de frameworks puissants comme Django, Flask ou FastAPI.

Le choix du tuteur d'imposer Python pour ce stage repose donc sur une vraie stratégie pédagogique : me faire évoluer dans un environnement proche de ce que recherchent aujourd'hui les recruteurs, tout en me fournissant une base technique solide pour les années à venir.

Afin de transformer Python en véritable application web, j'ai utilisé le framework Django. Son architecture MVT (Modèle–Vue–Template) m'a appris à séparer les données, la logique métier et l'affichage. J'ai pu tirer parti de nombreuses fonctionnalités intégrées à Django, qui ont considérablement accéléré le processus de développement. L'ORM (Object-Relational Mapper), notamment, m'a permis d'interagir avec la base de données en manipulant des objets Python, sans avoir à écrire manuellement des requêtes SQL.

De plus, des outils natifs comme le système d'authentification et les protections contre les failles de sécurité courantes (comme les attaques par injection SQL ou les CSRF) m'ont non seulement fait gagner un temps précieux, mais m'ont aussi sensibilisé aux bonnes pratiques de sécurité web.

Pour la partie visuelle du projet, le choix s'est porté sur Tailwind CSS, un framework que je découvrais pour la première fois. Contrairement aux bibliothèques plus traditionnelles comme Bootstrap, Tailwind adopte une approche dite "utility-first". Cela signifie qu'au lieu de créer ses propres classes CSS dans un fichier séparé, on applique directement des classes prêtes à l'emploi sur les balises HTML, pour gérer la mise en page, les couleurs, les espacements etc... Ce framework m'a aussi permis de mieux comprendre les bases du responsive design, puisque tout est pensé pour s'adapter aux différentes tailles d'écrans... Ce fut une vraie découverte, et un outil que je serai ravie de réutiliser dans mes prochains projets.

Enfin, j'ai utilisé Git et GitHub pour versionner le projet. Cette pratique m'a permis d'organiser mon travail, de garder un historique clair et de me familiariser avec un outil incontournable du monde professionnel.

2.3 Conception fonctionnelle : wireframes

Avant toute ligne de code, j'ai conçu des wireframes afin de poser la structure de l'application. L'objectif n'était pas de produire un design abouti, mais de clarifier les parcours utilisateurs et les informations à afficher.

Ce travail préparatoire, même s'il ne produit pas encore de code, m'a permis de coder plus vite, plus proprement et avec une vision d'ensemble. C'est une étape que je referai systématiquement dans mes projets futurs, même pour des applications simples.

Le formulaire de publication de recette, par exemple, m'a permis de visualiser la répartition des champs (titre, ingrédients, étapes, image) et d'anticiper leur correspondance avec les modèles Django. La page

d'accueil, quant à elle, a d'abord été pensée avec de nombreux filtres et options, mais je l'ai simplifiée au fur et à mesure du développement pour garder une navigation claire et intuitive. [Voir annexe 2 et 3.](#)

Ces modèles m'ont évité des hésitations techniques et ont servi de guide tout au long du développement. Même si le rendu final diffère des wireframes initiaux, ce travail préparatoire a structuré ma réflexion et accéléré la mise en œuvre.

2.4 Conception technique : schéma de base de données

En parallèle des wireframes, j'ai réalisé un schéma relationnel dans MySQL Workbench afin de définir la structure de la base de données.

[Voir annexe 4.](#) Quatre entités principales ont été retenues :

- User : représente les utilisateurs, avec identifiant, pseudo, email, mot de passe et informations complémentaires (âge, niveau, etc.).
- Recettes : contient les informations liées à chaque recette (titre, description, ingrédients, étapes, image, auteur).
- Commentaires : associent un texte et une date à une recette donnée, afin de favoriser l'interaction.
- Notes : enregistrent les évaluations attribuées par les utilisateurs, liées à la fois à une recette et à un utilisateur.

Les relations suivent une logique simple :

- Un utilisateur peut publier plusieurs recettes (relation **One-to-Many**).
- Une recette peut recevoir plusieurs commentaires (**One-to-Many**).

- Les notes établissent une relation plusieurs-à-plusieurs (many-to-many) entre les utilisateurs et les recettes, car un même utilisateur peut attribuer une note à plusieurs recettes, et une même recette peut recevoir des notes de plusieurs utilisateurs.

Cette modélisation a permis d'assurer la cohérence et l'intégrité des données (éviter les doublons de notes, empêcher des recettes orphelines, etc.). Grâce à l'ORM de Django, les relations définies dans mes modèles Python sont automatiquement traduites en clés étrangères dans la base de données. Cela me permet de manipuler les liens entre les entités, comme retrouver toutes les recettes d'un utilisateur, en travaillant directement avec des objets Python. Je n'ai donc pas eu besoin d'écrire de requêtes SQL complexes, ce qui a rendu le développement plus rapide et plus intuitif.

3. Mise en place du projet et développement technique

3.1 Structuration du projet Django

Dès le départ, j'ai choisi de structurer mon projet Django en plusieurs applications indépendantes plutôt que de tout centraliser dans un seul bloc. Dans Django, une application est un module qui regroupe un ensemble de fonctionnalités cohérentes : modèles (base de données), vues (logique), templates (interface), formulaires et URLs. Cette organisation respecte le principe de modularité recommandé par Django, rend le code plus lisible et facilite la maintenance à long terme.

J'ai donc mis en place deux applications principales :

- authentification, dédiée à la gestion des utilisateurs (inscription, connexion, déconnexion),

- recettes, qui constitue le cœur du projet et gère l'ensemble des fonctionnalités liées aux recettes, commentaires et notes.

Avant de développer ces fonctionnalités, j'ai pris le temps de mieux comprendre le cycle de traitement d'une requête dans Django, basé sur l'architecture MVT (Modèle – Vue – Template). Pour m'aider, j'ai réalisé une carte mentale retraçant le parcours complet : une requête envoyée par l'utilisateur est redirigée via `urls.py`, traitée par une vue qui interagit avec le modèle, puis transmise à un template pour l'affichage. Cette étape m'a permis d'avoir une vision claire du fonctionnement interne de Django et d'aborder le développement avec une méthode structurée. [Voir annexe 5.](#)

3.2 Développement des fonctionnalités principales

La première fonctionnalité que j'ai développée concernait la gestion des utilisateurs. Pour ce faire, j'ai créé un modèle utilisateur personnalisé en héritant de la classe `AbstractUser` de Django.

Cette approche m'a permis de bénéficier d'une base solide, qui inclut déjà des champs essentiels comme l'identifiant, l'e-mail et le mot de passe, ainsi qu'une gestion sécurisée des mots de passe et des permissions. J'ai pu ensuite ajouter facilement de nouveaux champs, comme l'âge, tout en conservant les fonctionnalités natives de Django. [Voir annexe 6.](#)

Ce modèle a ensuite été déclaré dans `settings.py` grâce à la variable `AUTH_USER_MODEL`. [Voir annexe 7.](#)

À partir de ce modèle, j'ai mis en place :

- L'inscription : un formulaire vérifiant les informations saisies et créant automatiquement un compte, suivi d'une connexion immédiate de l'utilisateur, [voir annexe 8](#).
- La connexion et la déconnexion, basées sur les méthodes natives `authenticate()` et `login()` qui garantissent une vérification sécurisée des identifiants. En effet, `authenticate()` valide les informations fournies et retourne l'objet utilisateur si elles sont correctes, tandis que `login()` établit la session, permettant à l'utilisateur de rester connecté sur le site. [voir annexe 9](#) et [19](#).
- Les routes correspondantes dans `urls.py`, afin de relier chaque action à la vue correspondante, [voir annexe 10](#).

Ces fonctionnalités ont ensuite été intégrées dans des templates conçus avec TailwindCSS, offrant une interface à la fois claire et responsive, [voir annexe 11, 12 et 13](#). Des messages d'erreur s'affichent en cas de saisie incorrecte, et la directive `{% csrf_token %}` protège les formulaires contre les attaques CSRF. Cette attaque est une faille de sécurité qui force un utilisateur authentifié à exécuter une action non désirée sur un site web. Pour s'en prémunir, Django utilise un jeton CSRF unique pour chaque session. La directive insère ce jeton dans le formulaire, et lorsque celui-ci est soumis, Django vérifie que le jeton reçu correspond bien à celui qu'il a généré. Si ce n'est pas le cas, la requête est rejetée, bloquant efficacement l'attaque.

Une fois l'authentification terminée, j'ai développé le cœur de l'application : la gestion des recettes (CRUD). J'ai défini un modèle `Recettes` comprenant

un titre, une description, des ingrédients, des étapes, une catégorie, un temps estimé, une image et une relation avec l'auteur via une clé étrangère vers User. Grâce au paramètre `on_delete=models.CASCADE`, la suppression d'un utilisateur entraîne automatiquement celle de ses recettes, évitant les données orphelines. [Voir annexe 14.](#)

Les principales fonctionnalités mises en place ont été :

- une page d'accueil listant toutes les recettes, [voir annexe 15.](#)
- un formulaire de création basé sur un `ModelForm`, avec `commit=False` pour ajouter automatiquement l'utilisateur connecté avant l'enregistrement, [voir annexe 16.](#)
- une page de détails affichant photo, ingrédients, étapes, commentaires et moyenne des notes, calculée via une fonction d'agrégation, [voir annexe 17.](#)
- la modification d'une recette en passant une instance existante au formulaire, [voir annexe 18.](#)

Enfin, pour rendre l'application agréable à utiliser, j'ai travaillé les templates : la page d'accueil affiche chaque recette sous forme de carte responsive, avec titre, catégorie et temps de préparation. Les actions de modification et de suppression ne s'affichent que pour l'auteur de la recette. Ce travail m'a permis de comprendre que le CRUD n'est pas seulement une logique de base de données, mais bien l'ossature d'une application dynamique où modèle, logique métier et interface s'articulent en continu. [Voir annexe 20, 21.](#)

3.3 Sécurité et intégrité des données.

Même si l'application n'était pas destinée à la production, je voulais adopter de bons réflexes de sécurité. J'ai donc mis en place plusieurs protections :

- Accès contrôlé : toutes les vues interactives sont protégées par le décorateur `@login_required`. Un décorateur est une fonction qui enveloppe une autre fonction pour lui ajouter des fonctionnalités. Dans ce cas précis, il vérifie si l'utilisateur est authentifié avant d'exécuter la vue, garantissant que seules les personnes connectées peuvent créer, modifier ou supprimer du contenu ([voir annexe 22](#)).
- Validation des fichiers : pour éviter les uploads malveillants, j'ai ajouté une fonction qui bloque tout fichier n'étant pas une image (PNG, JPG ou JPEG). En cas d'extension non autorisée, Django lève une `ValidationError` et l'upload est refusé, [voir annexe 23](#).
- Intégrité des notes : j'ai écrit un décorateur personnalisé (`one_rating_only`) qui empêche un utilisateur de noter plusieurs fois la même recette, afin de garantir des moyennes fiables [voir annexe 24](#).

Ces mesures illustrent deux dimensions de la sécurité : la protection contre des attaques techniques (ex. fichiers malveillants) et la garantie de la cohérence logique des données (ex. notation unique par utilisateur). Il est crucial de noter que cette protection ne peut pas se limiter au frontend. En effet, bien que la validation côté client (dans le navigateur) améliore l'expérience utilisateur, elle est facilement contournable. C'est pourquoi le backend doit impérativement re-valider toutes les informations reçues. Cette approche à double-validation garantit que les données qui sont finalement traitées et stockées dans la base de données sont à la fois

sécurisées et cohérentes, car on ne peut jamais faire entièrement confiance aux informations provenant du navigateur.

3.4 Frontend et expérience utilisateur

Une fois le backend terminé, j'ai travaillé sur l'interface pour transformer des fonctionnalités brutes en une application claire, esthétique et intuitive. Pour cela, j'ai utilisé TailwindCSS, un framework qui permet de styliser directement dans le HTML à l'aide de classes utilitaires. Cette approche m'a fait gagner du temps, tout en garantissant une cohérence graphique.

J'ai intégré Tailwind via la version CDN et personnalisé la palette de couleurs et la typographie, [voir annexe 25](#). Pour la barre de navigation, je me suis appuyée sur un composant existant que j'ai adapté aux besoins du projet, [voir annexe 26](#).

Les formulaires ont ensuite été retravaillés pour plus de lisibilité et de clarté : champs arrondis, ombres légères, focus coloré, [voir annexe 27](#). J'ai également ajouté des affichages conditionnels, comme l'apparition des boutons "Modifier" et "Supprimer" uniquement pour l'auteur de la recette, [voir annexe 28](#).

Le rendu final, responsive et homogène, s'éloigne des wireframes de départ pour proposer une interface plus vivante et professionnelle.

3.5 Gestion du code avec Git et Github

En parallèle du développement, j'ai travaillé avec Git et GitHub pour versionner mon projet. Cela m'a permis de comprendre que Git n'est pas seulement un outil de sauvegarde, mais un moyen de garder un historique

clair, de tester des fonctionnalités en parallèle et de travailler de façon professionnelle.

Avant ce stage, j'avais une vision assez simpliste : Git, c'était juste pour sauvegarder du code. J'ai vite compris que c'était bien plus que ça. Git permet de travailler proprement, revenir en arrière à tout moment et collaborer efficacement, même si dans mon cas j'étais seule sur le projet.

J'ai commencé par configurer l'authentification via SSH, puis j'ai créé mon dépôt distant et lié mon projet local, [voir annexes 29, 30 et 31](#). Une fois l'environnement prêt, j'ai adopté un workflow inspiré de GitFlow, adapté à un projet solo :

- main : version stable, prête à être présentée,
- dev : zone d'intégration, toutes les fonctionnalités sont testées ensemble
- feature : une branche par fonctionnalité, fusionnée ensuite dans dev.

Ce modèle a évité de fragiliser la branche principale et m'a appris à travailler de manière rigoureuse, en documentant et en testant chaque étape avant intégration. J'ai aussi découvert la légèreté et la souplesse des branches Git, qui permettent de passer rapidement d'une fonctionnalité à l'autre.

Au-delà du code, j'ai compris l'importance d'une bonne documentation. J'ai donc ajouté un README.md clair, avec la description du projet, les étapes d'installation et les prérequis. J'ai également généré un fichier requirements.txt pour simplifier la mise en place de l'environnement. Ces

deux fichiers rendent mon projet exploitable par n'importe quel développeur qui voudrait le cloner et le tester, *voir annexe [32,33](#)*.

3.6 Problèmes rencontrés et solutions

Au début, j'ai rencontré plusieurs difficultés avec Git.

- Lors de mon premier push, mon dépôt local était encore sur la branche master, alors que GitHub attendait main, *voir annexe [34](#)*. J'ai corrigé le problème en renommant la branche avec `git branch -M main`.
- Un autre blocage est survenu après avoir ajouté un README directement sur GitHub : les historiques local et distant divergeaient. J'ai résolu ce conflit avec `git pull origin main --allow-unrelated-histories`.
- Enfin, j'ai découvert les merge conflicts en fusionnant dev avec une branche feature. Après quelques essais, j'ai appris à les résoudre dans l'éditeur avant de valider la fusion, *voir annexe [35](#)*.

En parallèle de ces problèmes liés au versionnement, j'ai également rencontré des difficultés côté Django. Au départ, je filtrais les recettes avec l'id passé dans l'URL. Cela posait un problème de sécurité : en modifiant ce paramètre, un utilisateur pouvait accéder aux recettes d'un autre. J'ai corrigé cette logique en filtrant directement sur `request.user`, ce qui limite l'affichage aux recettes appartenant à la personne connectée, *voir annexe [36](#)*.

J'ai aussi rencontré une difficulté lors d'une **migration** après l'ajout d'un champ obligatoire dans le modèle Commentaire. Django refusait la migration car la table contenait déjà des données. Pour résoudre ce problème, j'ai rendu le champ facultatif (null=True, blank=True) avant d'appliquer la migration, [voir annexe 37](#).

4. Relations hiérarchiques et sociales

Mon stage a été encadré par Farouk, développeur principal de Visual Vibes. Bien que très sollicité, il a pris le temps de m'accompagner lorsque cela était nécessaire, notamment pour m'expliquer des notions complexes ou m'aiguiller dans mes blocages techniques. En dehors de ces moments clés, il m'a laissée travailler de manière autonome, ce qui m'a permis d'apprendre en expérimentant. J'ai bénéficié d'une vraie liberté dans mes choix techniques, tant sur la structuration de l'application que sur la manière de résoudre les problèmes rencontrés. Ce cadre autonome m'a permis de progresser rapidement, tout en étant rassurée par le soutien ponctuel de Farouk lorsque j'atteignais une impasse.

J'ai également pu prendre des initiatives, comme :

- l'adaptation des templates à mon projet,
- l'implémentation de certaines couches de sécurité,
- ou encore la gestion du front avec TailwindCSS.

Visual Vibes étant une micro-entreprise composée de deux personnes, mes interactions ont été limitées mais précieuses. J'ai surtout travaillé avec Farouk pour les aspects techniques, et avec Charlotte, webdesigner, pour

la direction artistique du site. Elle m'a aidée à clarifier l'identité visuelle et m'a orientée sur l'ergonomie.

Ce binôme complémentaire m'a permis d'évoluer dans une ambiance bienveillante, où la rigueur technique allait de pair avec une vraie attention au design.

J'ai également renforcé mes compétences en communication : savoir formuler clairement mes blocages, poser les bonnes questions, documenter mon code pour faciliter les échanges.

5. Conclusion et perspectives

Ce stage m'a permis de consolider mes bases en développement web, en travaillant concrètement avec Django, Git/GitHub et TailwindCSS. J'ai appris à structurer un projet complet en plusieurs applications, à gérer les modèles, vues et templates, et à mettre en place des relations cohérentes entre les données.

Côté back-end, j'ai approfondi l'authentification, le CRUD, les décorateurs et la gestion des requêtes. Côté front-end, j'ai transformé des wireframes en interfaces fonctionnelles avec une approche responsive et accessible. J'ai également acquis une méthode de travail plus professionnelle : versionner mon code proprement, organiser mes fichiers, tester mes fonctions, documenter mes choix et anticiper les erreurs.

Au-delà de l'aspect technique, ce stage m'a permis de gagner en autonomie, en confiance et en capacité à formuler mes questions. Les échanges réguliers avec mon tuteur m'ont aidée à progresser sur des

sujets que je n'aurais pas abordés seule et m'ont confortée dans ma capacité à mener un projet web complet de bout en bout.

5.1 Prolongement du stage

Une fois l'application de recettes finalisée, j'ai poursuivi avec un mini-projet complémentaire en console : une application de gestion de budget.

L'objectif était de travailler ma logique algorithmique tout en découvrant l'architecture orientée services.

Cette application permet à l'utilisateur de créer des catégories de dépenses, d'y associer des dépenses, de les modifier, de les filtrer et de visualiser des statistiques comme le total dépensé. Ce projet m'a permis de revenir aux fondamentaux (fonctions, classes, conditions, manipulation de dates) tout en adoptant une organisation rigoureuse du code, sans interface graphique.

5.2 Perspectives d'avenir

Ce stage m'a donné envie d'approfondir certains aspects du développement. À court terme, je souhaite renforcer mes compétences en programmation orientée objet avec Python et explorer davantage la sécurisation des applications web, notamment avec Django.

À plus long terme, j'aimerais découvrir l'univers du DevOps et du déploiement en production (CI/CD, conteneurisation, gestion des environnements), afin de mieux comprendre le cycle de vie complet d'une application.

Enfin, ce stage m'a confortée dans mon choix de carrière : je veux continuer à évoluer dans le développement web, en alliant rigueur technique et souci de l'expérience utilisateur.

Annexes :

1)

Classement des langages les plus utilisés.

Source: <https://www.tiobe.com/tiobe-index>

Jun 2025	Jun 2024	Change	Programming Language	Ratings	Change
1	1		 Python	25.87%	+10.48%
2	2		 C++	10.68%	+0.65%
3	3		 C	9.47%	+0.24%
4	4		 Java	8.84%	+0.44%
5	5		 C#	4.69%	-1.96%
6	6		 JavaScript	3.21%	-0.11%

2)

Wireframe du formulaire de publication de recette



Publish your own recipes

Name of your recipe

Prep Time

Servings

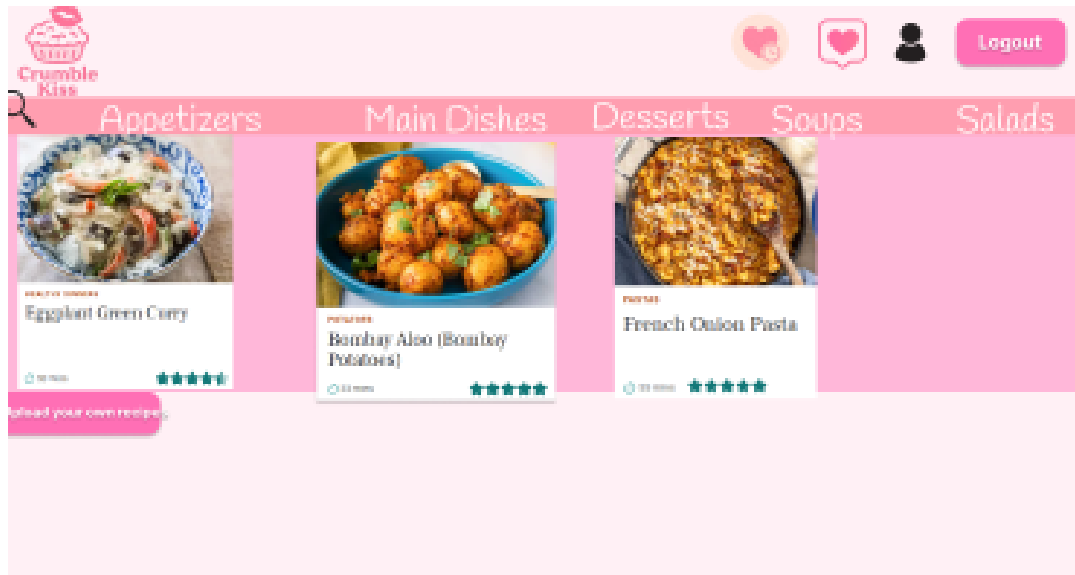
Ingredients

Description

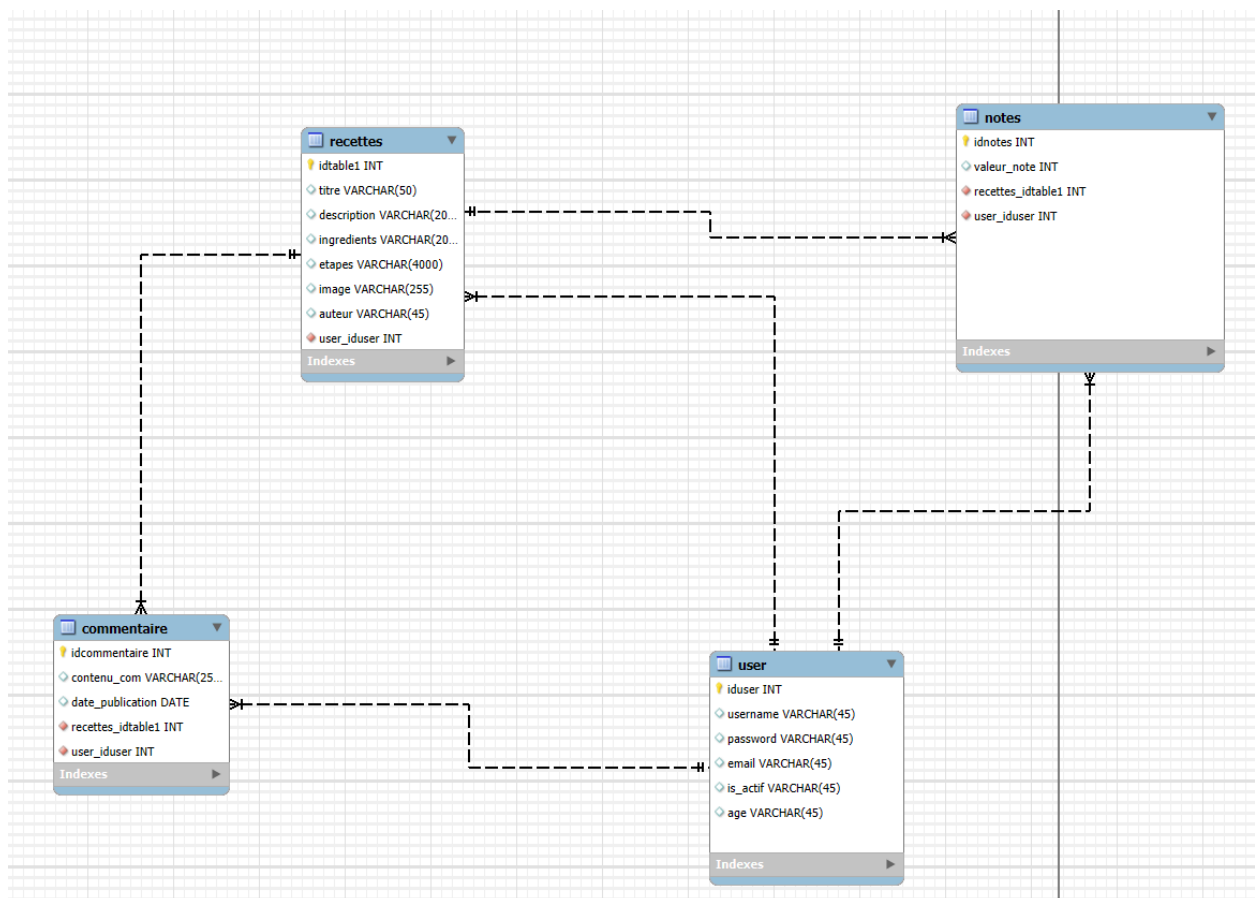
Publish

3)

Wireframe de la page d'accueil et affichage des recettes

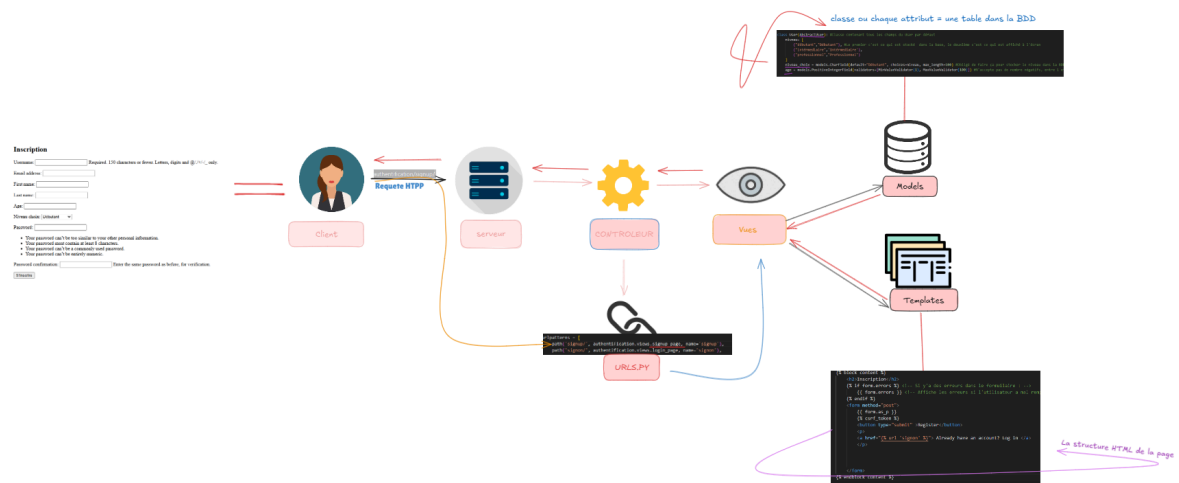


4) Présentation du modèle de données.



5)

Schéma pour comprendre le modèle MVT.



6)

Modèle utilisateur

```

class User(AbstractUser):
    niveau= [
        ("débutant", "Débutant"),
        ("intermédiaire", "Intermédiaire"),
        ("professionnel", "Professionnel")
    ]
    niveau_choix = models.CharField(default="Débutant", choices=niveau, max_length=100)
    age = models.PositiveIntegerField(validators=[MinValueValidator(1), MaxValueValidator(100)])
  
```

7)

Préciser à Django quel modèle utiliser pour ne pas qu'il utilise le sien par défaut.

```

AUTH_USER_MODEL = 'authentification.User'
  
```

8)

Fonction permettant à l'utilisateur de s'inscrire via le formulaire.

```
def signup_page(request):
    form = forms.SignupForm()
    if request.method == 'POST':
        form = forms.SignupForm(request.POST)
        if form.is_valid():
            user = form.save() #enregistre utilisateur en bdd et stocke
            # auto-login user
            login(request, user)
            return redirect('home_page')
    if request.method == "GET":
        return render(request, 'signup.html', context={'form': form})
```

9)

Utilisation de la méthode authenticate pour comparer les informations entrées à ceux en base de données.

```
if form.is_valid():
    user = authenticate(
        username=form.cleaned_data['username'],
        password=form.cleaned_data['password'],
    )
```

10)

Rendre accessible les vues grâce au routing

```
urlpatterns = [
    path('signup/', authentication.views.signup_page, name='signup'),
    path('signon/', authentication.views.login_page, name='signon'),
    path('logout/', authentication.views.log_out, name='logout'),
]
```

11)

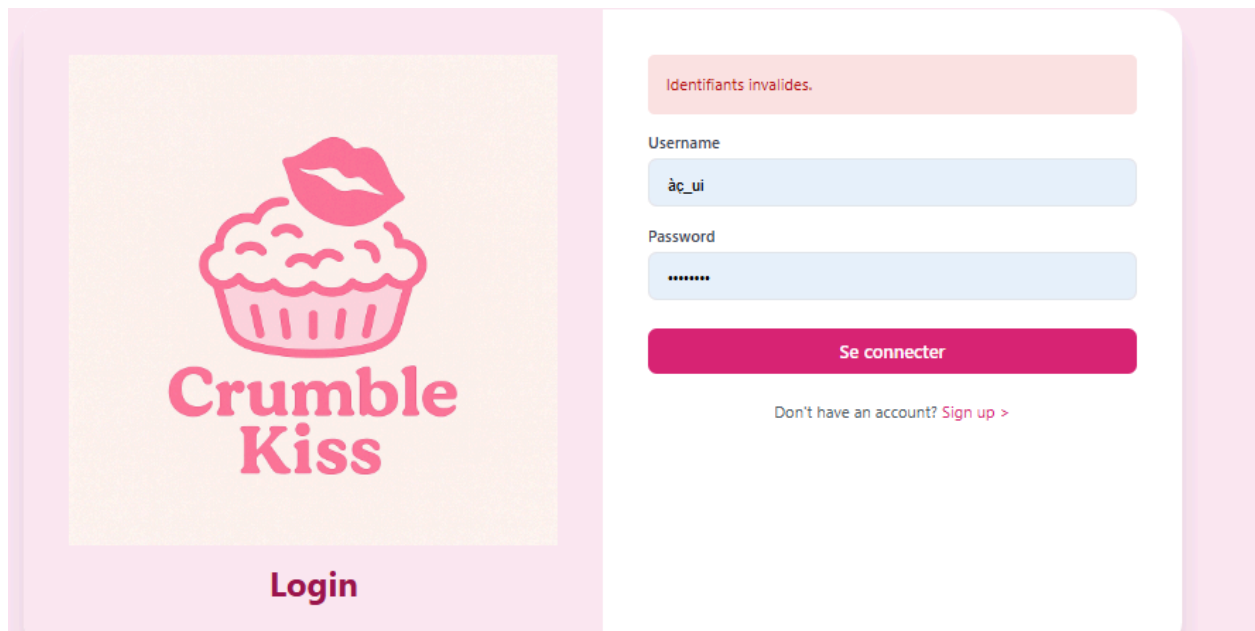
Extrait du template de connexion

```
<!-- Champ Username -->
<div>
  <label for="{{ form.username.id_for_label }}" class="block text-sm font-medium text-gray-700 mb-1">
    Username
  </label>
  {{ form.username }}
</div>

<!-- Champ Password -->
<div>
  <label for="{{ form.password.id_for_label }}" class="block text-sm font-medium text-gray-700 mb-1">
    Password
  </label>
  {{ form.password }}
</div>
```

12)

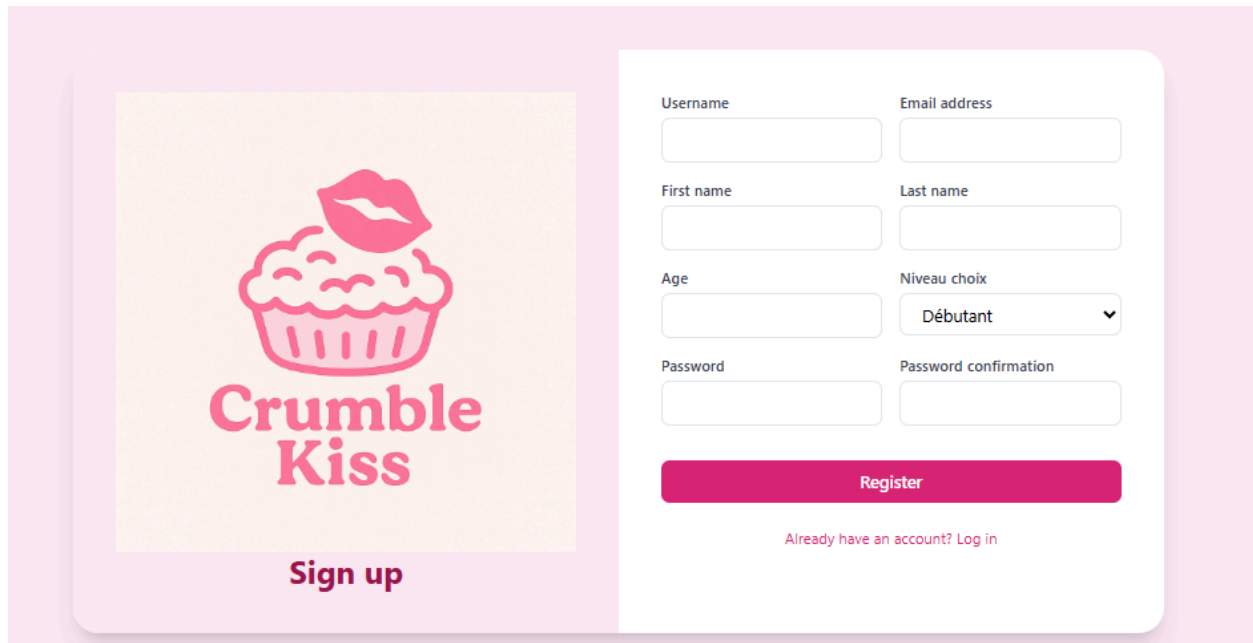
Page de connexion avec tailwind CSS



The image shows a login page for 'Crumble Kiss'. On the left, there is a logo featuring a pink cupcake with a kiss mark on top, with the text 'Crumble Kiss' in a pink, rounded font below it. Underneath the logo is a pink button labeled 'Login'. On the right, there is a white login form with a pink border. At the top of the form is a pink error message: 'Identifiants invalides.'. Below this are two input fields: 'Username' and 'Password'. The 'Username' field contains the text 'àç_ui'. The 'Password' field is filled with dots. Below the input fields is a pink button labeled 'Se connecter'. At the bottom of the form, there is a link: 'Don't have an account? [Sign up >](#)'.

13)

Page d'inscription



The registration page for 'Crumble Kiss' features a pink-themed design. On the left, a card displays a cupcake logo with a pink lip on top, the text 'Crumble Kiss' in a bold, pink font, and a 'Sign up' button. On the right, a white registration form is set against a light pink background. The form includes fields for Username, Email address, First name, Last name, Age, and Password, along with a dropdown menu for 'Niveau choix' (Level choice) currently set to 'Débutant' (Beginner). A 'Register' button is at the bottom of the form, and a link 'Already have an account? Log in' is positioned below it.

14)

Modèle de données utilisés pour la création de recettes

```
class Recettes(models.Model):
    id = models.AutoField(primary_key=True)
    recipe_title = models.CharField(max_length=60)
    short_description = models.CharField(max_length=400)
    ingredients_list = models.TextField(blank=True)
    preparation_steps = models.TextField(blank=True)
    category_choices = [
        ("appetizers", "Appetizers"),
        ("main dishes", "Main Dishes"),
        ("soups", "Soups"),
        ("salads", "Salads"),
        ("desserts", "Desserts")
    ]

    category_choices = models.CharField(default="", choices=category_choices, max_length=60)
    time_in_minutes = models.PositiveIntegerField(default=0, validators=[MinValueValidator(0), MaxValueValidator(300)])
    user = models.ForeignKey(User, on_delete=models.CASCADE)
```

15)

Vue permettant d'afficher les recettes sur la page d'accueil

```
def home_page(request):
    all_recette= Recettes.objects.all()
    context = {
        'recettes' : all_recette,
    }
    return render(request, 'homepage.html', context)
```

16)

Vue permettant la création de recettes via un formulaire par les utilisateurs

```
def create_recipes(request):
    form = forms.Creation()
    file_not_supported = False
    if request.method == "POST":
        form = forms.Creation(request.POST)
        if form.is_valid():
            recipe = form.save(commit=False)
            recipe.user = request.user
            pictures = request.FILES["filename"]
            recipe.save()
```

17) Vue permettant d'afficher les détails des recettes, les commentaires ainsi que les notes et une moyenne de celle-ci.

```
@login_required
def recipe_details(request, id):
    recette = get_object_or_404(Recettes, id=id)
    commentaires = recette.commentaires_set.all()
    notes = recette.notes_set.all()

    note_limite = notes.filter(user=request.user)
    if note_limite:
        user_have_rating = True
    else:
        user_have_rating = False
    total = 0
    for note in notes :
        total+= note.valeur_notes
    if len(notes) == 0:
        moyenne = 0
    else :
        moyenne = round(total/len(notes),1)
    print(recette.id)
    return render(request, 'recipes_details.html', {'recette': recette, 'commentaires': commentaires, 'not
```

18)

Vue permettant aux utilisateurs de modifier leurs recettes.

```
@login_required
def recipe_update(request, id_from_url):
    recette = get_object_or_404(Recettes, id=id_from_url, user=request.user)
    if request.method == "GET":
        form = forms.Creation(instance=recette)
        return render(request, 'recipes_creation.html', {'form': form} )
    if request.method == "POST":
        form = forms.Creation(request.POST, instance=recette)
        if form.is_valid():
            recipe = form.save()
            return redirect('home_page')
```

19)

Vue permettant aux utilisateurs de se déconnecter.

```
@login_required
def recipe_delete(request, id_from_url):
    mes_recette = get_object_or_404(Recettes, id=id_from_url, user=request.user)
    if request.method == "POST":
        mes_recette.delete()

    return redirect('my_recipes')
```

20)

Exemple du template de l'accueil

```
{% block content %}
<div class="mt-8 grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-6">
  {% for recette in recettes %}
    <details class="bg-white rounded-2xl shadow-lg p-6 transition-all hover:shadow-xl open:shadow-xl">
      <summary class="list-none">
        <h2 class="text-2xl font-bold text-gray-800 mb-3">{{ recette.recipe_title }}</h2>

        <div class="flex items-center gap-2 text-sm text-gray-600 mb-1">
          <span class="text-pink-500">🕒</span>
          <span>{{ recette.time_in_minutes }} minutes</span>
        </div>

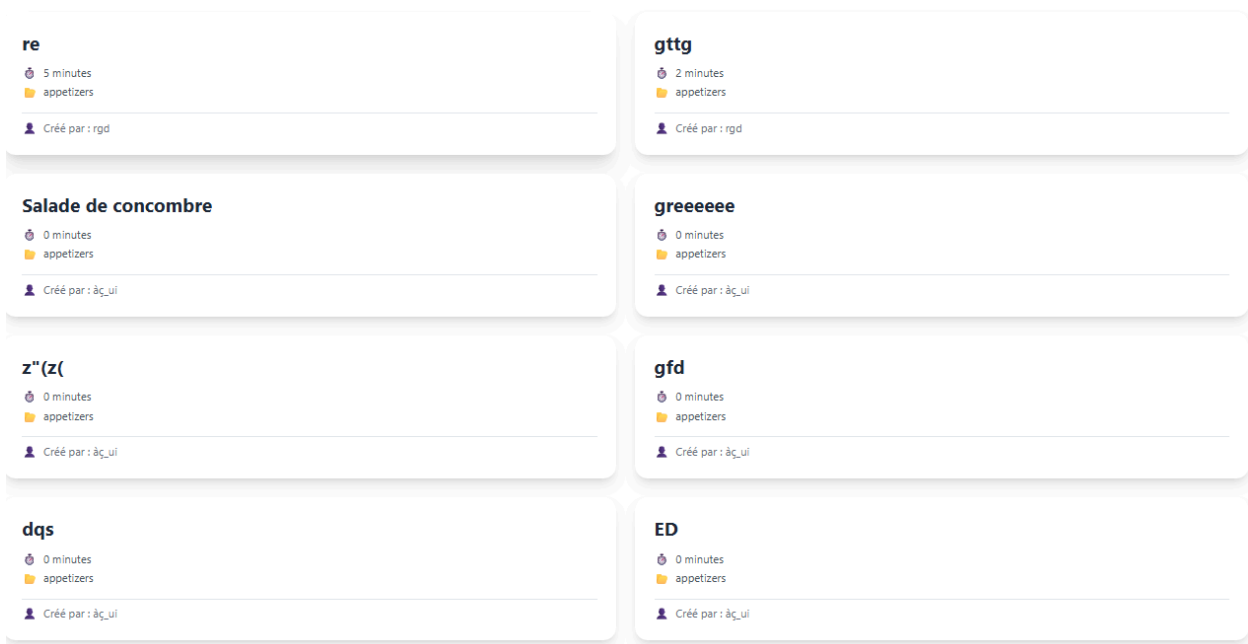
        <div class="flex items-center gap-2 text-sm text-gray-600 mb-1">
          <span class="text-blue-500">🍽️</span>
          <span>{{ recette.category_choices }}</span>
        </div>

        <div class="flex items-center gap-2 text-sm text-gray-500 mt-4 pt-2 border-t">
          <span class="text-green-500">👤</span>
          <span>Créé par : {{ recette.user.username }}</span>
        </div>
      </summary>

      <!-- Aperçu visible uniquement quand la carte est ouverte -->
    </details>
  {% endfor %}
</div>
```

21)

Résultat de l'interface de l'accueil après une modification du front end.



22)

Utilisation du décorateur fourni par Django : @login_required

```
@login_required #Peut pas y accéder si t'es pas
def home_page(request):
    all_recette= Recettes.objects.all()
    context = {
        'recettes' : all_recette,
    }
    return render(request, 'homepage.html', con

@login_required
def recipe_update(request, id_from_url):
    recette = get_object_or_404(Recettes,id=id_from_u
    if request.method == "GET":
        form = forms.Creation(instance=recette)
        return render(request, 'recipes_creation.html', {'form': for
    if request.method == "POST":
        form = forms.Creation(request.POST, instance=recette)
        if form.is_valid():
            recipe = form.save()
            return redirect('home_page')

@login_required
def rating(request,recette_id):
    recette = Recettes.objects.get(id=recette_id)

    if request.method == "POST":
        form = Notes(request.POST)
        if form.is_valid():
            notes = form.save(commit=False)
            notes.recettes = recette
            notes.user = request.user
            notes.save()
            return redirect("home_page")
    return render(request, "recipes_detail.htm

@login_required
def recipe_delete(request, id_from_url):
    mes_recette = get_object_or_404(Re
    if request.method == "POST": #Pas
        mes_recette.delete()

@login_required
def add_comments(request, recette_id): #i
    recette = Recettes.objects.get(id=rec

    if request.method == "POST":
        form = Comments(request.POST)
        if form.is_valid():
            commentaire = form.save(commi
            commentaire.recettes = recett

@login_required
def recipe_details(request, id):
    recette = get_object_or_404(Recettes, i
    commentaires = recette.commentaires_set
    notes = recette.notes_set.all()

    note_limite = notes.filter(user=request
    if note_limite:
        user_have_rating = True
    else:
        user_have_rating = False
```

23)

Vue permettant de restreindre le type de fichier à uploader

```
def validate_files(value):
    if value.content_type != "image/png" and value.content_type != "image/jpg" and value.content_type != "image/jpeg":
        raise ValidationError('File not supported')
```

24)

Décorateurs pour empêcher aux utilisateurs de noter plusieurs fois.

```
from functools import wraps
from django.shortcuts import get_object_or_404, redirect

from .models import Recettes

def one_rating_only(view_func):
    @wraps(view_func)
    def _wrapped(request, *args, **kwargs):
        id = kwargs.get('recette_id')
        recette = get_object_or_404(Recettes, id=id)
        notes = recette.notes_set.all()
        note_limite = notes.filter(user=request.user)
        if note_limite:
            return redirect("home_page")
        return view_func(request, *args, **kwargs)
    return _wrapped
```

25)

Utilisation de tailwind via le <head>.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Publish your own recipes</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <script>
    tailwind.config = {
      theme: {
        extend: {
          colors: {
            primary: '#ec4899',
            pastel: '#ffe4e6',
            chocolate: '#4b2c2c'
          },
          fontFamily: {
            sans: ['"Poppins"', 'sans-serif']
          }
        }
      }
    }
  </script>
</head>
```

26)

Extrait de code pour la barre de navigation donnée par tailwind.

```
<nav class="bg-navbg border-b border-pink-200 shadow-sm">
  <div class="max-w-5xl mx-auto px-4 sm:px-6 lg:px-8">
    <div class="flex justify-between h-16 items-center">
      <!-- Logo & Brand -->
      <div class="flex items-center space-x-4">
        
        <span class="text-2xl font-bold text-accent">Crumble Kiss</span>
      </div>

      <!-- Liens -->
      <div class="hidden md:flex space-x-8 text-sm font-medium text-chocolate">
        <a href="{% url 'home_page' %}" class="hover:text-accent">All Recipes</a>
        <a href="{% url 'create_recipes' %}" class="hover:text-accent">Add a Recipe</a>
        <a href="{% url 'my_recipes' %}" class="hover:text-accent">My recipes</a>
      </div>

      <!-- Utilisateur -->
      <div class="relative ml-3">
        <div>
          <button type="button" class="relative flex rounded-full bg-gray-800 text-sm focus:ring-2 focus:ring-white focus:ring-of
            <span class="absolute -inset-1.5"></span>
            <span class="sr-only">Open user menu</span>
            <!-- Avatar neutre sans image -->
            <div class="size-8 rounded-full bg-gray-400 flex items-center justify-center text-white text-xs font-bold">
              ?
            </div>
          </div>
        </button>
      </div>
    </div>
  </div>
```

27)

Formulaire pour publier une recette

28)

Possibilité de modifier ou de supprimer la recette uniquement si on est le créateur.

```
{% if source == "my_recipes" %}
    <a href="{% url 'update_recipes' recette.id %}" class="text-blue-600 underline mt-2 inline-block">Modifier la recette

    <form method="POST" action="{% url 'delete_recipes' recette.id %}">
{% csrf_token %}
<button type="submit" class="text-red-600 underline mt-2 inline-block">Delete recipe</button>
{% endif %}
```

29)

Ajout de la bonne clé à l'agent et enregistrée sur github.

```
PS C:\Users\melin> ssh -T git@github.com
Hi marissalmr! You've successfully authenticated, but GitHub does not provide shell access.
```

30)

Initialisation du dépôt git.

```
PS C:\stage\recette> git init
Initialized empty Git repository in C:/stage/recette/.git/
```

31)

Ajout des premiers fichiers.

```
PS C:\stage\recette> git add .
PS C:\stage\recette> git commit -m "Hello World page"
[master (root-commit) d13f8c9] Hello World page
22 files changed, 227 insertions(+)
create mode 100644 db.sqlite3
create mode 100644 manage.py
create mode 100644 recette/__init__.py
create mode 100644 recette/__pycache__/__init__.cpython-313.pyc
create mode 100644 recette/__pycache__/settings.cpython-313.pyc
create mode 100644 recette/__pycache__/urls.cpython-313.pyc
create mode 100644 recette/__pycache__/wsgi.cpython-313.pyc
create mode 100644 recette/asgi.py
create mode 100644 recette/settings.py
create mode 100644 recette/urls.py
create mode 100644 recette/wsgi.py
create mode 100644 recettes/__init__.py
```

32)

Aperçu du fichier requirements.txt

```
1 asgiref==3.8.1
2 Django==5.1.7
3 django-widget-tweaks==1.5.0
4 sqlparse==0.5.3
5 tzdata==2025.2
```

33)

Aperçu du fichier [README.MD](#)



34)

Erreur lors de la première tentative de pousser les fichiers

```
PS C:\stage\recette> git push -u origin main
error: src refspec main does not match any
error: failed to push some refs to 'github.com:marissalnr/recettes.git'
PS C:\stage\recette> git branch
* master
```

35)

Erreur lorsque j'ai coché l'option pour ajouter un fichier [README.MD](#).

```
PS C:\stage\recette> git pull origin main
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 879 bytes | 87.00 KiB/s, done.
From github.com:marissalmr/recettes
 * branch          main          -> FETCH_HEAD
 * [new branch]    main          -> origin/main
fatal: refusing to merge unrelated histories
```

36)

Correction de la vue pour la fonctionnalité de filtrage des données.

```
def my_recipes(request):
    print(request.user)
    mes_recettes = Recettes.objects.filter(user=request.user)
    return render(request, 'my_recipes.html', {'mes_recettes': mes_recettes})
```

37)

Erreur lors de l'ajout d'un nouveau champs après la création du modèle.

```
It is impossible to add the field 'date_publication' with 'auto_now_add=True' to commentaires without providing a default. This is because the database needs something to populate
1) Provide a one-off default now which will be set on all existing rows
2) Quit and manually define a default value in models.py.
select an option: 1
```